Introduction to Computer Architecture and Systems

Chapter 4
Programming in LC-3

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

Previously of CSCI 260…

Universal Concepts

- Computer Architecture and Computer Organization
- Memory Hierarchy
- Instruction Cycle
- Addressing Mode
- “Turing Complete”

Previously of CSCI 260…

Little Computer - 3: Introduction and ISA

- ‘Physical’ Hardware Resources
  - Registers, Memories, Programming Counter(Instruction Pointer)
  - Instruction Cycle and Addressing Mode
- LC-3 Instruction Set
  - Computation/Operation Instruction
  - Memory Instruction
  - Control Instruction

Instruction Set

15 Opcodes

- Operate instructions: **ADD**, **AND**, **NOT**
- Data movement instructions: **LD**, **LDI**, **LDR**, **LEA**, **ST**, **STR**, **STI**
- Control instructions: **BR**, **JSR/JSRR**, **JMP**, **RTI**, **TRAP**

- All definitions are in Chapter 5 of the textbook, available in Moodle.
- Complete the rest of the picture for yourself. When in doubt, contact me.

Cover-ups…

A Real Application for Amdahl's Law

Comparing Apple A16 and Apple A17 pro

- A17 pro performs 10% better in Single Core than A16
- A17 pro performs 20-30% better in GPU than A16
- For my program…
  - Rule of thumb, 10% of total codes are sequential(can not do in parallel)
  - CPU contributes 40% of bottleneck, GPU contributes 50% of bottleneck
  - How would be my performance improve from A16 to A17 pro?

$$\text{Speedup}_{\text{overall}} = \frac{1}{(1 - \text{time}_{\text{optimized}}) + \frac{\text{time}_{\text{optimized}}}{\text{speedup}_{\text{optimized}}}}$$

Addressing Modes

How is the location of an operand specified?

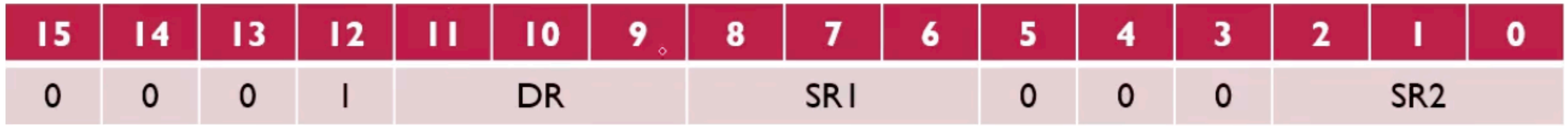
- non-memory addresses:
  - immediate
  - register
- memory addresses:
  - PC-relative
  - indirect
  - base+offset

Instructions of LC-3, Examples.

ADD

Opcode: 0001

- Destination Register(DR) = Source Register 1(SR1) + Source Register 2(SR2)



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	DR			SR1			0	0	0	SR2		

- Destination Register(DR) = Source Register 1(SR1) + 5-bit Immediate Number

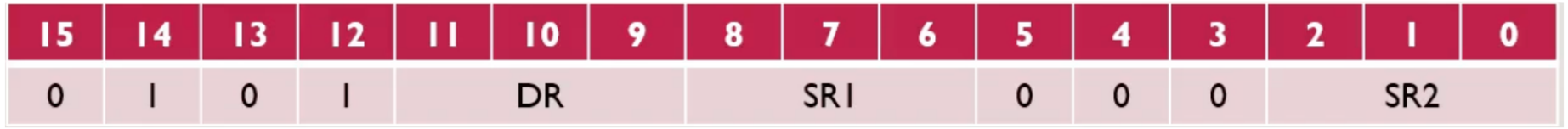


15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	DR			SR1			1	Imm5				

AND / &

Opcode: 0101

- Destination Register(DR) =
  Source Register 1(SR1) AND Source Register 2(SR2)



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	DR			SR1			0	0	0	SR2		

- Destination Register(DR) =
  Source Register 1(SR1) AND 5-bit Immediate Number

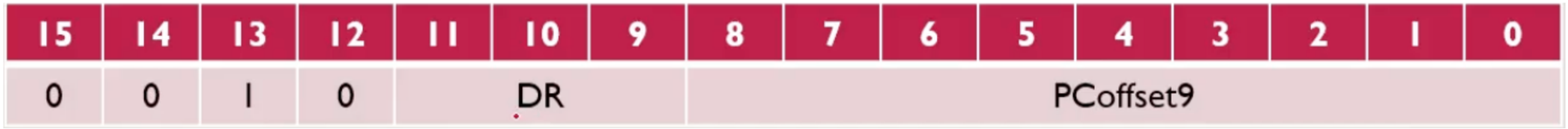


15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	DR			SR1			1	Imm5				

LD

Opcode: 0010

- Destination Register(DR) =
  the content of memory <PCoffset9> lines away from the Program Counter



BR(Branch)

Opcode: 0000

- If (the output from ALU gave us a Negative / Zero / Positive result), we set the PC to <PCoffset9> of our current position

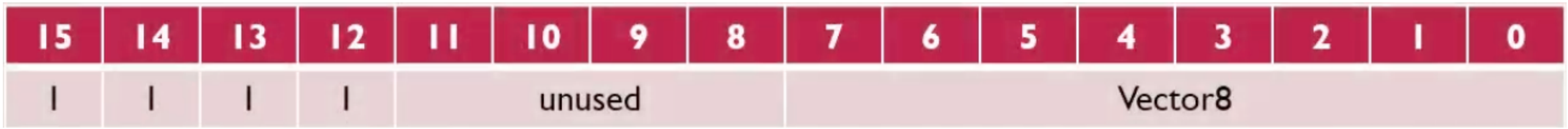
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	Cond3, a.k.a. N | Z | P			PCoffset9								

N	Z	P	Value is…
0	0	0	
0	0	1	
0	1	0	= 0
0	1	1	
1	0	0	
1	0	1	!= 0
1	1	0	
1	1	1	

TRAP

Opcode: 1111

- “Run program at the index of [Vector8]”



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	unused				Vector8							

- Program at x0025 is the ‘HALT’ program.

Opcode		Vector8
1111	0000	0010 0101
1111 0000 0010 0101		
xF025		

= x0025

= HALT the processor

More Instructions

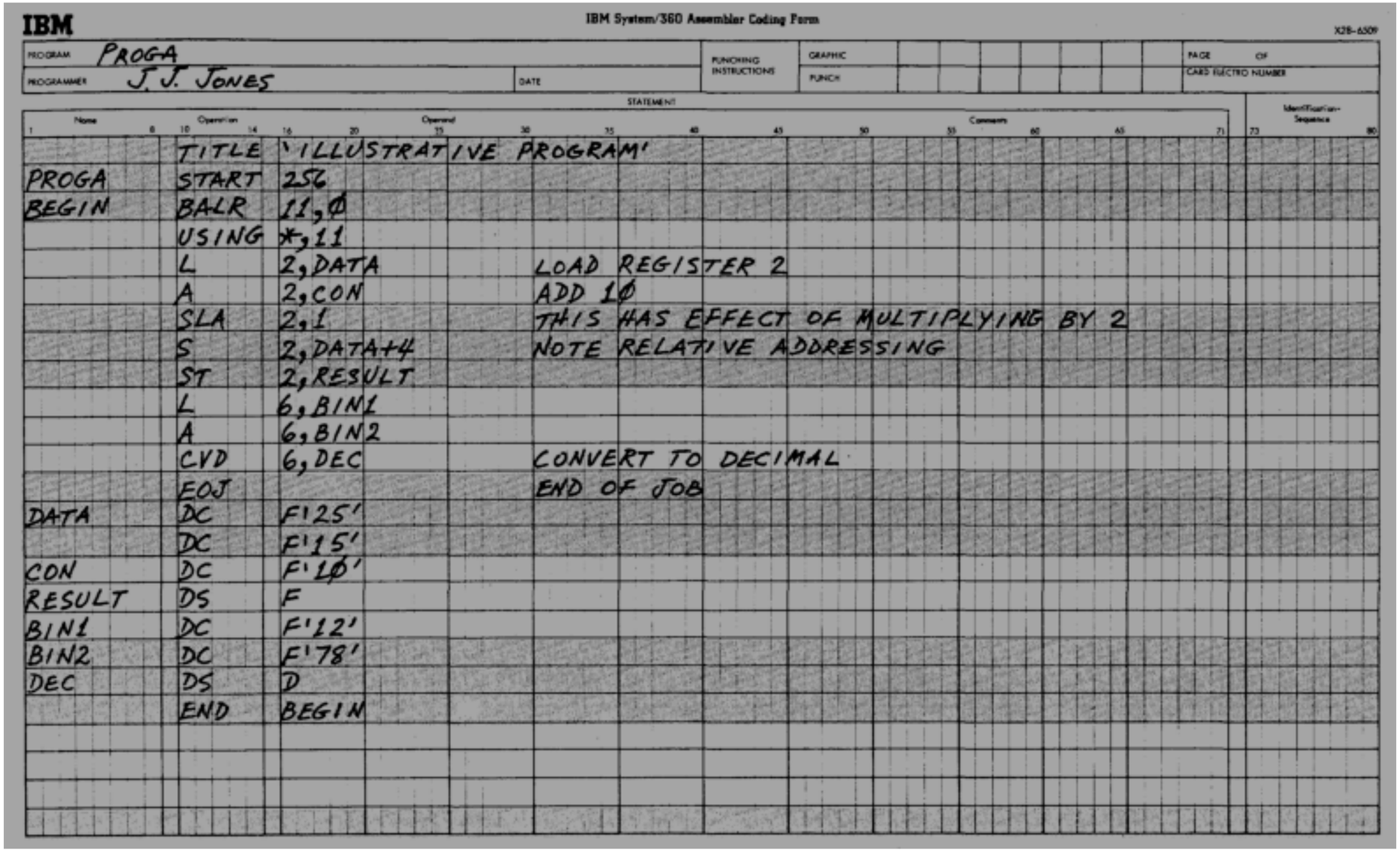
Operate	Data Movement	Control
ADD	LD	BR
AND	**LDI**	TRAP
NOT	**LDR**	**JMP**
LEA	**ST**	
	STI	
	STR	

Assignment:
Complete your surviving package of LC-3 and Assembly

Work Sheet: ‘Coding Form’

Programming in Assembly

With IBM S/360



IBM System/360 Assembler Coding Form — X28-6509

PROGRAM: PROGA
PROGRAMMER: J. J. JONES
DATE:
PUNCHING INSTRUCTIONS — GRAPHIC / PUNCH
PAGE OF
CARD ELECTRO NUMBER

STATEMENT

Name	Operation	Operand	Comments
	TITLE	'ILLUSTRATIVE PROGRAM'	
PROGA	START	256	
BEGIN	BALR	11,Ø	
	USING	*,11	
	L	2,DATA	LOAD REGISTER 2
	A	2,CON	ADD 1Ø
	SLA	2,1	THIS HAS EFFECT OF MULTIPLYING BY 2
	S	2,DATA+4	NOTE RELATIVE ADDRESSING
	ST	2,RESULT	
	L	6,BIN1	
	A	6,BIN2	
	CVD	6,DEC	CONVERT TO DECIMAL
	EOJ		END OF JOB
DATA	DC	F'25'	
	DC	F'15'	
CON	DC	F'1Ø'	
RESULT	DS	F	
BIN1	DC	F'12'	
BIN2	DC	F'78'	
DEC	DS	D	
	END	BEGIN	

Our Coding Form for LC-3

Feel free to add more elements of your own…

Date: Program's Name:

CSCI 260 Coding Form: LC-3 Assembly

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Mnemonics	Notes

- Available on Moodle!
- Get physical copies!
- Make your own!

Building Survival Kit

I’ll check your draft this Friday~

- Conversion Table between Binaries, Decimals, Hexadecimal.
- Quick lookup tables for ASCII
- Structure of LC-3 (Datapath Diagram)
- 15 Instructions and Examples
- State Trackers (for Memory and Registers)
- Coding Form(Done!)

Our First Program: Multiplication

Workflow

Always think first… always not hurt with a pen first.

- Problem Identify and Breakdown(Top-Down)
- Transfer into Assembly, once granularity reach to ISA level
- Write out mnemonic and check if it works, with State Tracker
- Translate into binary code or hexadecimal code(Bottom-Up)
- Write back to the machine and run

Write Assembly in 5-Steps

Problem Identify and Breakdown

A x B is “A add A, for B times…”

-> storage for A and B

A need to add A

-> another storage for result

count B times

-> state change for B

-> Condition with B to Halt the multiplication

A x B is “A add A, for B times…”

-> storage for A and B -> R1 and R2

A need to add A

-> another storage for result -> R3

count B times

-> state change for B -> B - 1, until B = 0

-> Condition with B to Halt the multiplication

A x B is “A add A, for B times…”

-> storage for A and B -> R1 and R2

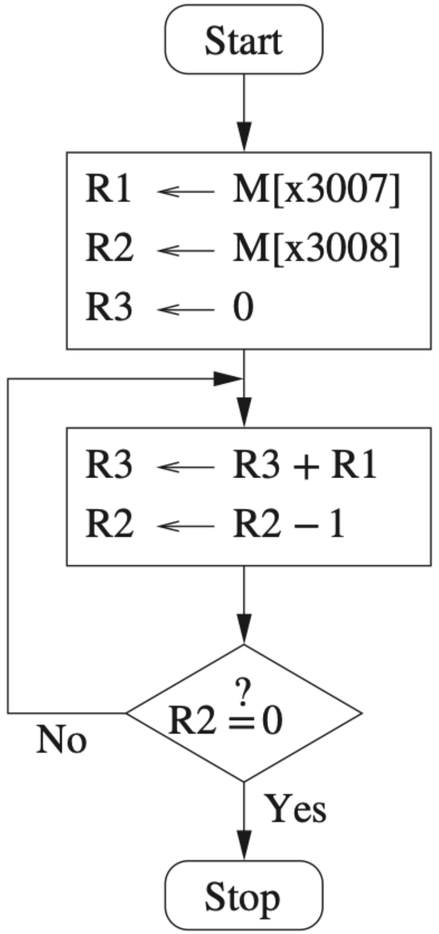
A need to add A

-> another storage for result -> R3

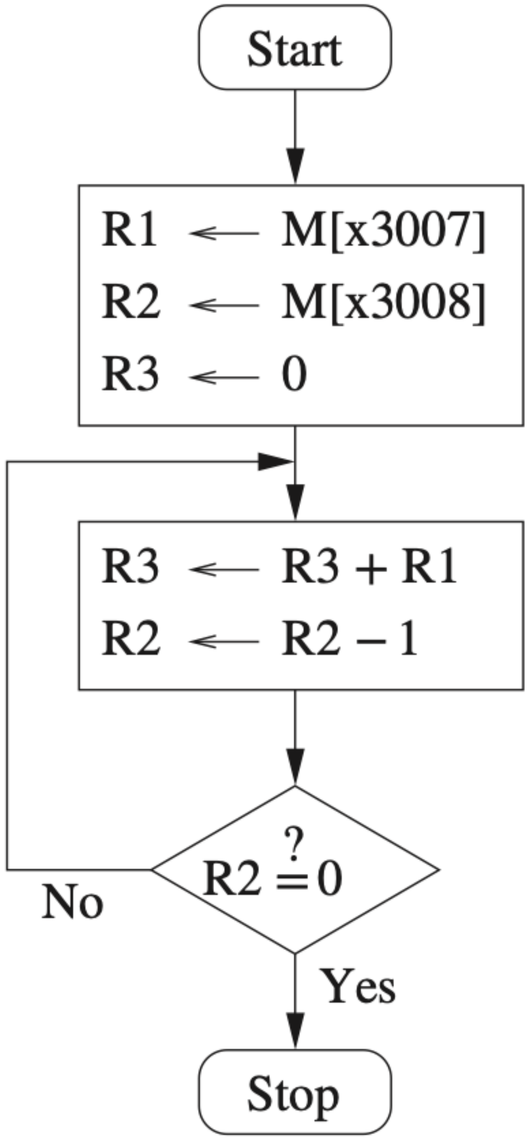
count B times

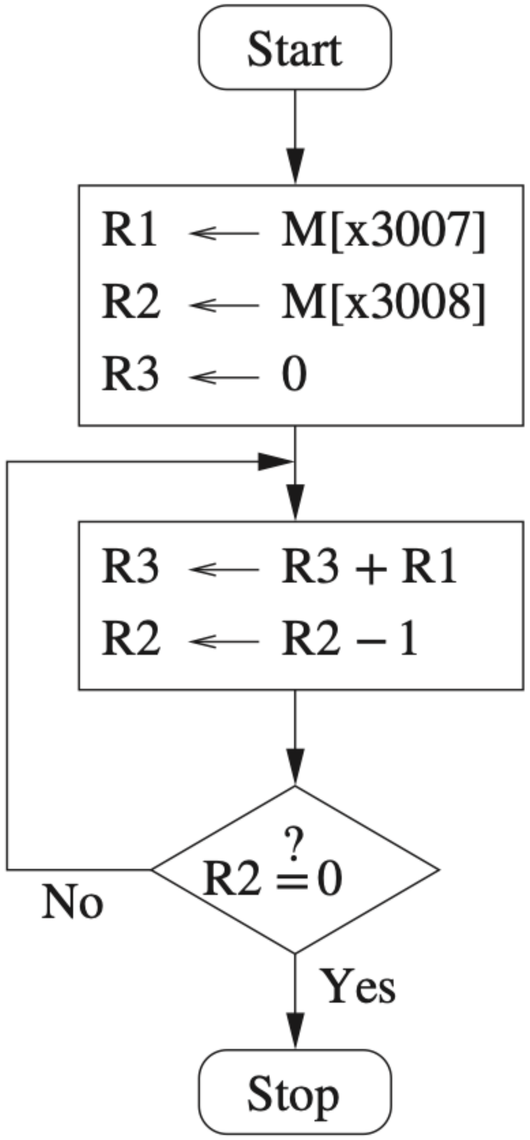
-> state change for B -> B - 1, until B = 0

-> Condition with B to Halt the multiplication



Transfer into Assembly


Start
R1 ← M[x3007]
R2 ← M[x3008]
R3 ← 0
R3 ← R3 + R1
R2 ← R2 − 1
R2 ?= 0
No
Yes
Stop

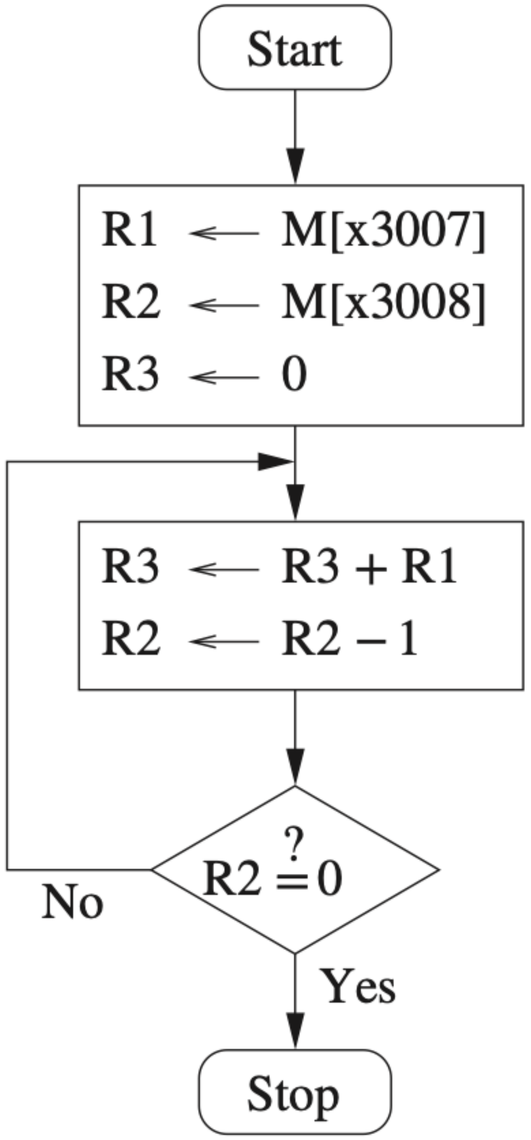


LD
LD
LD

ADD
ADD

BR

TRAP(HALT)



LD
LD
LD

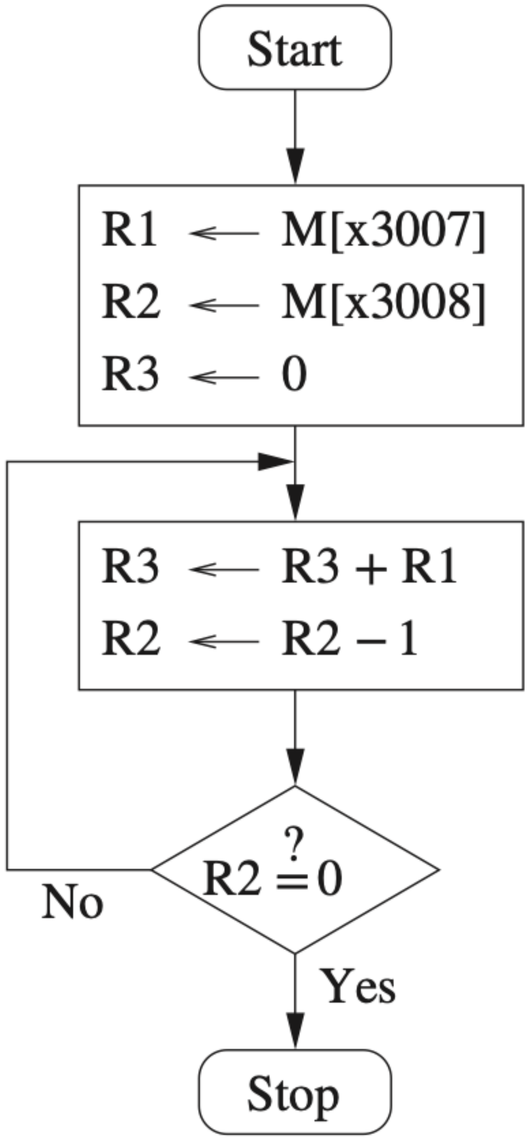
ADD
ADD

BR

TRAP(HALT)

```
R1 <- M[x3007]
R2 <- M[x3008]
R3 <- 0
R3 <- R3+R1
R2 <- R2-1
BR not-zero M[x3003]
HALT
```

Check with State Tracker



LD
LD
LD

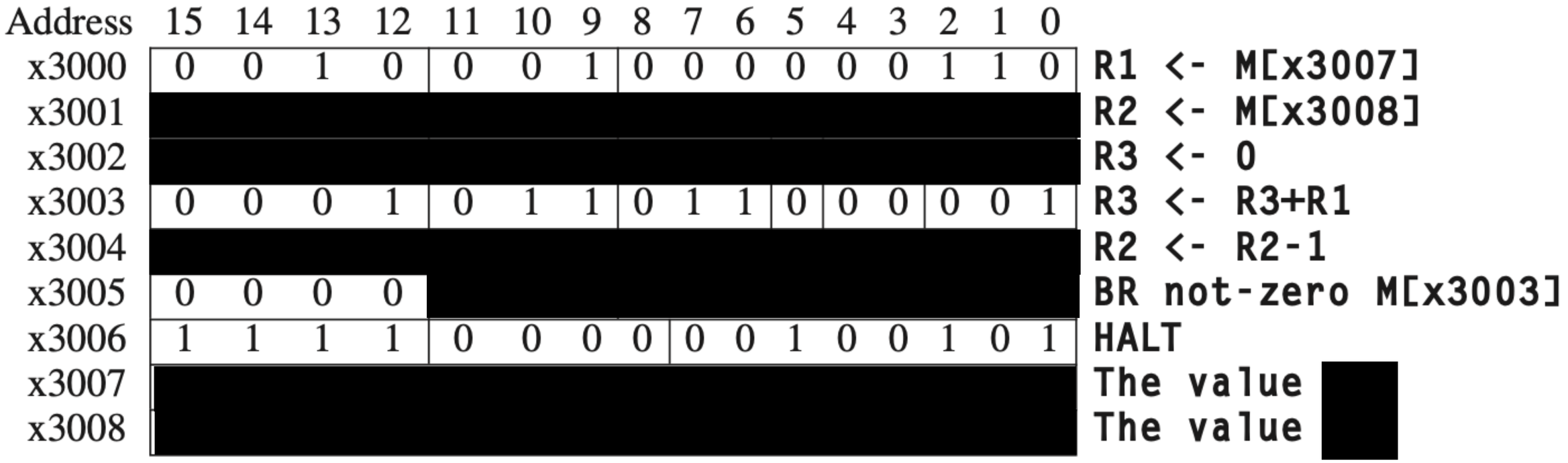
ADD
ADD

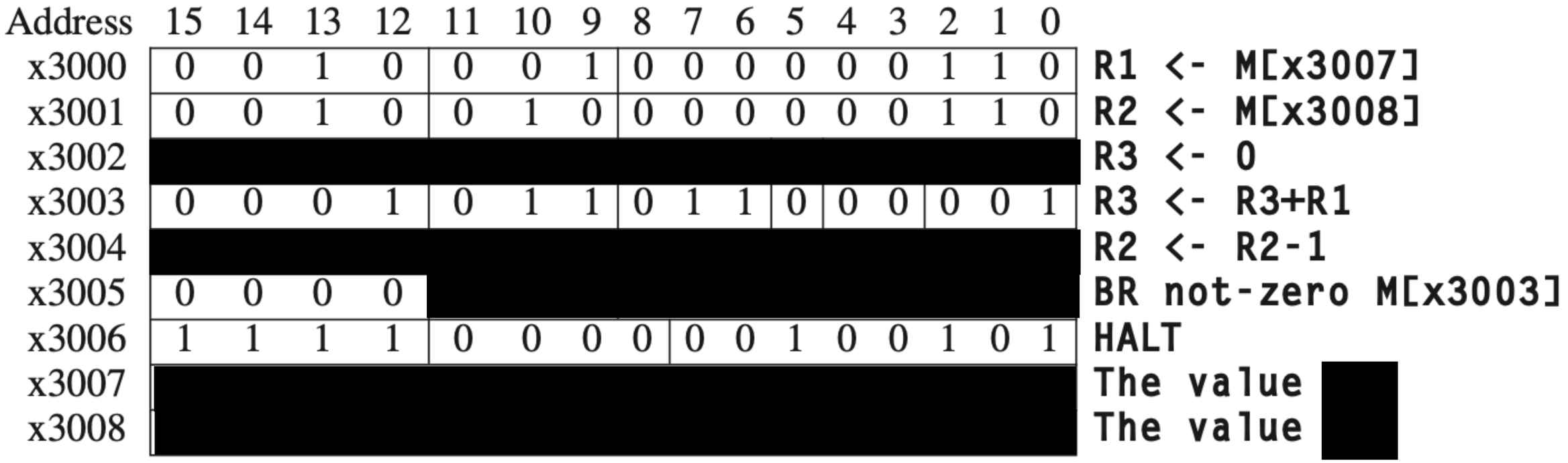
BR

TRAP(HALT)

```
R1 <- M[x3007]
R2 <- M[x3008]
R3 <- 0
R3 <- R3+R1
R2 <- R2-1
BR not-zero M[x3003]
HALT
```

Translate into code


Address 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
x3000 0 0 1 0 0 0 1 0 0 0 0 0 0 1 1 0
R1 <- M[x3007]
x3001
R2 <- M[x3008]
x3002
R3 <- 0
x3003 0 0 0 1 0 1 1 0 1 1 0 0 0 0 0 1
R3 <- R3+R1
x3004
R2 <- R2-1
x3005 0 0 0 0
BR not-zero M[x3003]
x3006 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1
HALT
x3007
The value
x3008
The value


Address 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
x3000 0 0 1 0 0 0 1 0 0 0 0 0 0 1 1 0 R1 <- M[x3007]
x3001 0 0 1 0 0 1 0 0 0 0 0 0 0 1 1 0 R2 <- M[x3008]
x3002 R3 <- 0
x3003 0 0 0 1 0 1 1 0 1 1 0 0 0 0 0 1 R3 <- R3+R1
x3004 R2 <- R2-1
x3005 0 0 0 0 BR not-zero M[x3003]
x3006 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1 HALT
x3007 The value
x3008 The value

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	0	1	0	0	0	1	0	0	0	0	0	0	1	1	0	R1 <- M[x3007]
x3001	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	0	R2 <- M[x3008]
x3002	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 <- 0
x3003	0	0	0	1	0	1	1	0	1	1	0	0	0	0	0	1	R3 <- R3+R1
x3004	[redacted]																R2 <- R2-1
x3005	0	0	0	0	[redacted]												BR not-zero M[x3003]
x3006	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT
x3007	[redacted]																The value [redacted]
x3008	[redacted]																The value [redacted]

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	0	1	0	0	0	1	0	0	0	0	0	0	1	1	0	R1 <- M[x3007]
x3001	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	0	R2 <- M[x3008]
x3002	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 <- 0
x3003	0	0	0	1	0	1	1	0	1	1	0	0	0	0	0	1	R3 <- R3+R1
x3004	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	R2 <- R2-1
x3005	0	0	0	0	[redacted]												BR not-zero M[x3003]
x3006	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT
x3007	[redacted]																The value [redacted]
x3008																	The value

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	0	1	0	0	0	1	0	0	0	0	0	0	1	1	0	R1 <- M[x3007]
x3001	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	0	R2 <- M[x3008]
x3002	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 <- 0
x3003	0	0	0	1	0	1	1	0	1	1	0	0	0	0	0	1	R3 <- R3+R1
x3004	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	R2 <- R2-1
x3005	0	0	0	0	1	0	1	1	1	1	1	1	1	1	0	1	BR not-zero M[x3003]
x3006	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT
x3007	[redacted]																The value [redacted]
x3008																	The value

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	0	1	0	0	0	1	0	0	0	0	0	0	1	1	0	R1 <- M[x3007]
x3001	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	0	R2 <- M[x3008]
x3002	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 <- 0
x3003	0	0	0	1	0	1	1	0	1	1	0	0	0	0	0	1	R3 <- R3+R1
x3004	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	R2 <- R2-1
x3005	0	0	0	0	1	0	1	1	1	1	1	1	1	1	0	1	BR not-zero M[x3003]
x3006	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT
x3007																	The value 5
x3008																	The value 4

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	0	1	0	0	0	1	0	0	0	0	0	0	1	1	0	R1 <- M[x3007]
x3001	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	0	R2 <- M[x3008]
x3002	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 <- 0
x3003	0	0	0	1	0	1	1	0	1	1	0	0	0	0	0	1	R3 <- R3+R1
x3004	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	R2 <- R2-1
x3005	0	0	0	0	1	0	1	1	1	1	1	1	1	1	0	1	BR not-zero M[x3003]
x3006	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT
x3007	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	The value 5
x3008																	The value 4

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	0	1	0	0	0	1	0	0	0	0	0	0	1	1	0	`R1 <- M[x3007]`
x3001	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	0	`R2 <- M[x3008]`
x3002	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	`R3 <- 0`
x3003	0	0	0	1	0	1	1	0	1	1	0	0	0	0	0	1	`R3 <- R3+R1`
x3004	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	`R2 <- R2-1`
x3005	0	0	0	0	1	0	1	1	1	1	1	1	1	1	0	1	`BR not-zero M[x3003]`
x3006	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	`HALT`
x3007	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	`The value 5`
x3008	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	`The value 4`

Write to machine and run

Cover-ups…

Write a subtract snippet…

So you will know why 2's complement, and why no SUB.

- Breakdown the Problem…
- Convert to Function…
- Rewrite to Assembly…
- Run on State Tracker…
- Write Code and Run…

Write a subtract snippet…

So you will know why 2's complement, and why no SUB.

- Breakdown the Problem…
  - Flip the bits
  - Add one
  - Perform add
- Convert to Function…
- Rewrite to Assembly…
- Run on State Tracker…
- Write Code and Run…

Write a subtract snippet…

So you will know why 2's complement, and why no SUB.



- Could this snippet cause problems?
- How to evade the issue?

Instruction Set

15 Opcodes

- Operate instructions: **ADD, AND, NOT**
- Data movement instructions: **LD, LDI, LDR, LEA, ST, STR, STI**
- Control instructions: **BR, JSR/JSRR, JMP, RTI, TRAP**

- *some opcodes set/clear condition codes, based on result:
- N = negative, Z = zero, P = positive (> 0)

LEA

Opcode: 1110, 'Load Effective Address'



- "1110" + Destination Register + <PC9Offset>
- Get PC and Offset, and write back to Destination Register

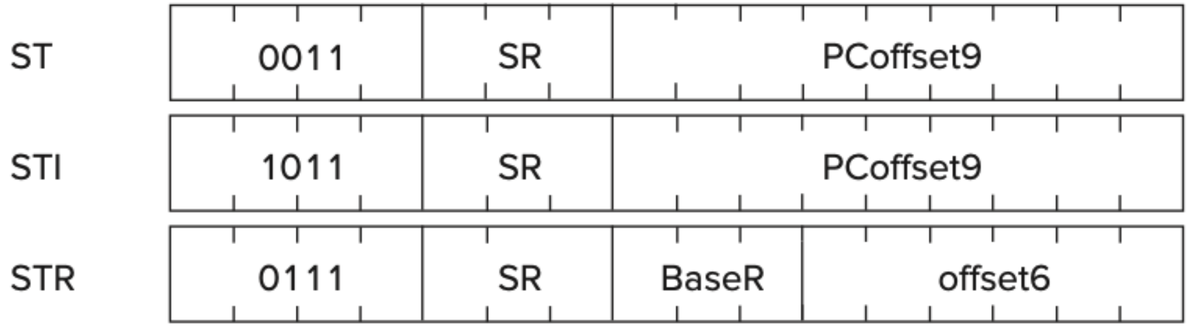
If memory location x4018 contains the instruction LEA R5, #−3, and the PC contains x4018,

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	1	1	1	1	1	1	1	0	1
LEA				R5			−3								

R5 will contain x4016 after the instruction at x4018 is executed. Question: Why will R5 not contain the address x4015?

ST

Opcode 0011, ’Store Data back to Memory’

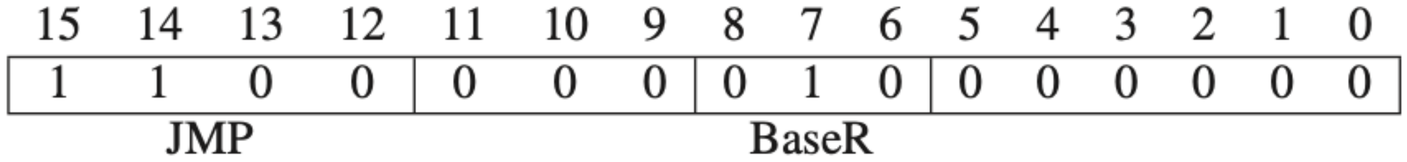


- Like LD, you can get corresponding positions of destination memory address by PC, or the base register.
- Base Register gives more freedom than counting from PC. You can see that again when discussing JMP.

JMP

Opcode: 1100, ’Unconditional Move, More Freely’

x3000



- If R2 has x6600, PC is at x3000,
- then, after executing JMP, PC will be at x6600.
- Considering BR has 9 digits of signed binaries, it can only shift -255 ~+256;
- JMP removes the limitation by sending 16-bit data via register.

ISA: Types of Instruction

- 1. Operate Instructions
  - process data (addition, logical operations, etc.)
- 2. Data Movement Instructions …
  - move data between memory locations and registers.
- 3. Control Instructions …
  - change the sequence of execution of instructions in the stored program.
    - The default is sequential execution: the PC is incremented by 1 at the start of every Fetch, in preparation for the next one.
    - Control instructions set the PC to a new value during the Execute phase, so the next instruction comes from a different place in the program. This allows us to build control structures such as loops and branches.

New Program: Counting Occurrence of a Character

Add-on for TRAP

New preloaded programs…

- x23 = Input a character from keyboard
- x21 = Output a character to the monitor
- x25 = Halt the program

Counting Occurrence of a Character

Before starting…

- New Sentinel: b00000100 = End of Transmission(EOT) from ASCII.
- Counting occurrence of a character = find matches in ASCII values until EOT.
- Break down the problem:
  - How to break down the problem so subproblems reaches to ISA level?
  - How to organize the control flow?
  - How to initialize values?
  - How to read the word and display the result?

Don’t forget your survival kit…

To lookup ASCII and ISAs

E.2 Standard ASCII codes

Table E.2 The Standard ASCII Table

ASCII			ASCII			ASCII			ASCII		
Character	Dec	Hex	Character	Dec	Hex	Character	Dec	Hex	Character	Dec	Hex
nul	0	00	sp	32	20	@	64	40	'	96	60
soh	1	01	!	33	21	A	65	41	a	97	61
stx	2	02	"	34	22	B	66	42	b	98	62
etx	3	03	#	35	23	C	67	43	c	99	63
eot	4	04	$	36	24	D	68	44	d	100	64
enq	5	05	%	37	25	E	69	45	e	101	65
ack	6	06	&	38	26	F	70	46	f	102	66
bel	7	07	'	39	27	G	71	47	g	103	67
bs	8	08	(	40	28	H	72	48	h	104	68
ht	9	09	)	41	29	I	73	49	i	105	69
lf	10	0A	*	42	2A	J	74	4A	j	106	6A
vt	11	0B	+	43	2B	K	75	4B	k	107	6B
ff	12	0C	'	44	2C	L	76	4C	l	108	6C
cr	13	0D	-	45	2D	M	77	4D	m	109	6D
so	14	0E	.	46	2E	N	78	4E	n	110	6E
si	15	0F	/	47	2F	O	79	4F	o	111	6F
dle	16	10	0	48	30	P	80	50	p	112	70
dc1	17	11	1	49	31	Q	81	51	q	113	71
dc2	18	12	2	50	32	R	82	52	r	114	72
dc3	19	13	3	51	33	S	83	53	s	115	73
dc4	20	14	4	52	34	T	84	54	t	116	74
nak	21	15	5	53	35	U	85	55	u	117	75
syn	22	16	6	54	36	V	86	56	v	118	76
etb	23	17	7	55	37	W	87	57	w	119	77
can	24	18	8	56	38	X	88	58	x	120	78
em	25	19	9	57	39	Y	89	59	y	121	79
sub	26	1A	:	58	3A	Z	90	5A	z	122	7A
esc	27	1B	;	59	3B	[	91	5B	{	123	7B
fs	28	1C	<	60	3C	\	92	5C	|	124	7C
gs	29	1D	=	61	3D	]	93	5D	}	125	7D
rs	30	1E	>	62	3E	^	94	5E	~	126	7E
us	31	1F	?	63	3F	_	95	5F	del	127	7F

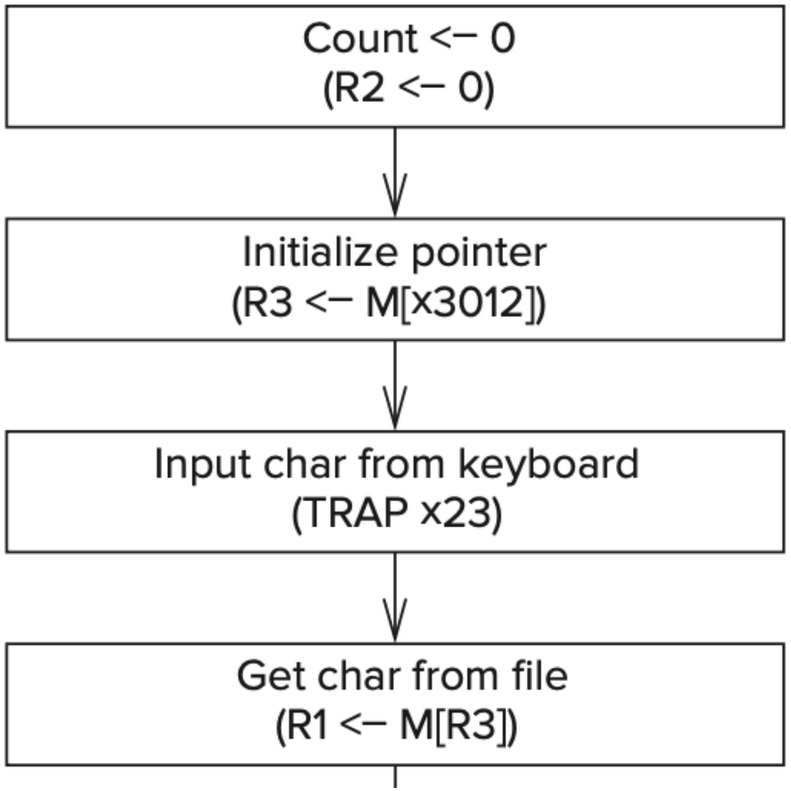
Counting Occurrence of a Character

Determining algorithm / solution

- Breaking down the problem
- Assigning resources
- Draw flowchart

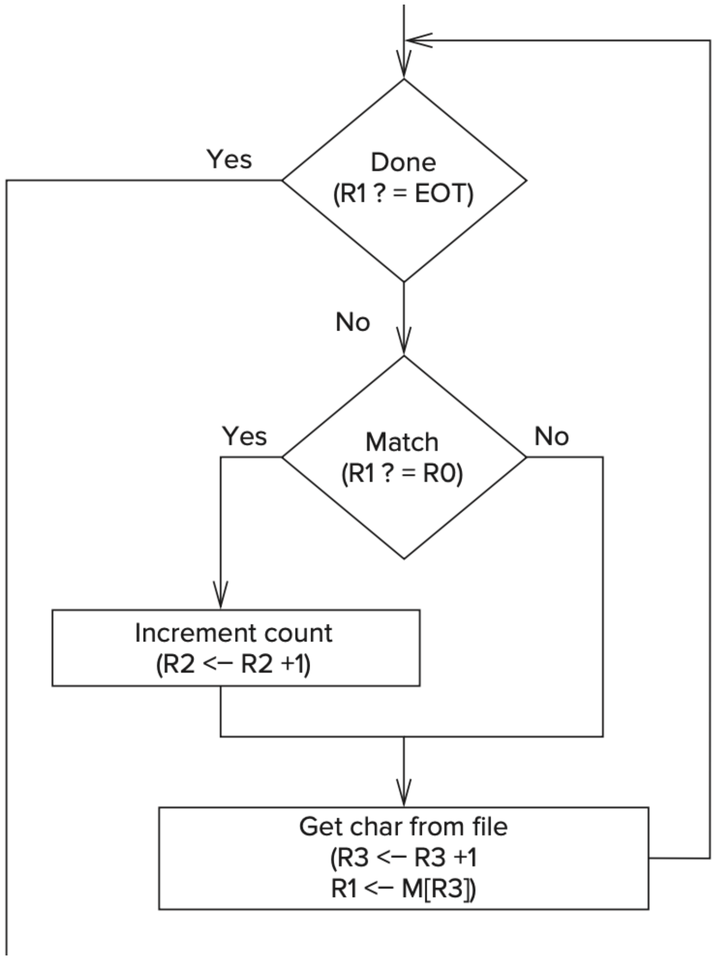
Counting Occurrence of a Character

Part 1: Get Input; Initialize Counter



Counting Occurrence of a Character

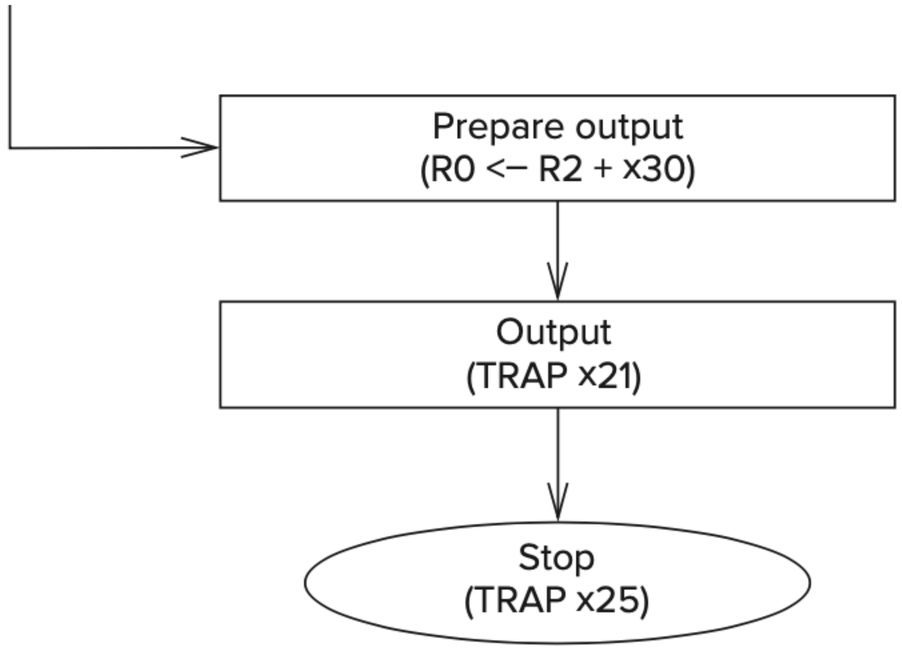
Part 2: Process inputs; counting matches



Counting Occurrence of a Character

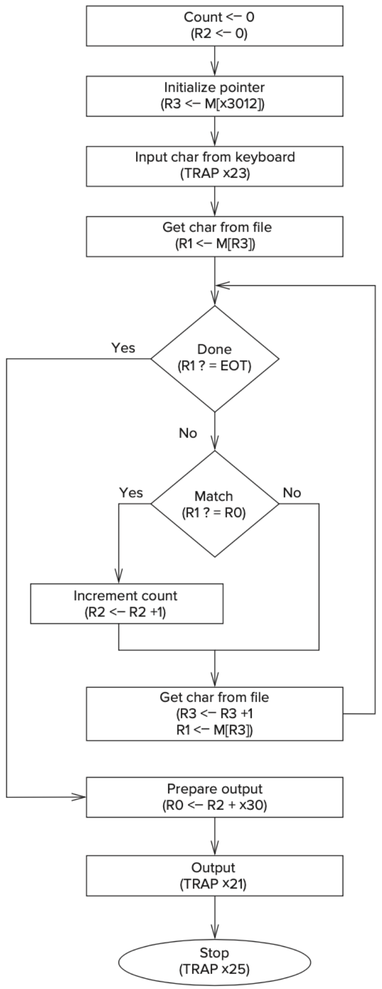
Part 3: Prepare Outputs, Finishing Up

If we reached EOT…



Counting Occurrence of a Character

Big Picture, then assign corresponding resources…



How many registers do we need?

How to call the loops?

How to make branches?

Next-Up: Control Flows